

The utility of rapid application development in large-scale, complex projects

Hilary Berger* & Paul Beynon-Davies†

*University of Wales Institute Cardiff, Cardiff School of Management, Colchester Avenue, Cardiff, CF23 9XR, UK, email: hberger@uwic.ac.uk, and †Cardiff Business School, Cardiff University, Aberconway Building, Colum Drive, Cardiff, CF10 3EU, UK, email: beynon-daviesp@cardiff.ac.uk

Abstract. *This paper describes an investigation into the utility of an agile development approach known as rapid application development (RAD) within a large-scale development project conducted within the public sector in the UK. Features of RAD as an 'agile' information systems development method (ISDM) are discussed and previous research identifying properties of development environments most conducive to its application are described. A case study is then presented based upon a long-term qualitative investigation of the 'unbundling' of a commercial RAD-like ISDM. The evidence from this case leads us to question established wisdom in relation to appropriate adoption and application of agile development within large-scale and complex development environments.*

Keywords: ISDM, agile development, large-scale development, RAD, case study

INTRODUCTION

Truex *et al.* (1999) identify dynamic requirements negotiation and continuous redevelopment as key goals of information systems development (ISD) in organizations subject to constant adjustment and adaptation. They state that, 'since organisational change has become so important to organisational survival, IT [information technology] systems must also incorporate continuous change' (p. 123). Due to the increasingly diverse nature of information systems (IS) and the ways in which they are applied, traditional 'heavyweight' approaches based upon linear development have been regarded as unsuited to the dynamic nature of much current ISD, particularly those projects focused on Web-based or 'Internet-speed' development. In contrast, more recent 'lightweight' approaches based upon an evolutionary and incremental life cycle are believed to be more suitable for these arenas (Baskerville & Pries-Heje, 2004; Kautz *et al.*, 2007). Baskerville *et al.* (2007) refer collectively to these approaches as agile development and propose that these new methods offer a significant alternative to plan-based (heavyweight) approaches.

This paper considers the application of a particular information systems development method (ISDM) within industrial practice. Rapid application development (RAD) is an ISDM

that bears a family resemblance to development approaches within the agile or lightweight camp in terms of features such as user involvement and iterative development. Previous researchers, such as those cited above, suggest that the successful application of a particular ISDM depends upon the degree of fit between the characteristics of the particular method and the characteristics of the development environment in which it is applied (McConnell, 1996; Hirschberg, 1998; Coughlan & Macredie, 2002). RAD as an approach has been seen as particularly applicable within small/medium-sized projects for the development of IS with relatively low complexity and a high degree of interactivity (Martin, 1991; Jones & King, 1998; Boehm, 1999). Hence, RAD has been applied frequently for the construction of web sites or for Web-based interfaces to back-end infrastructure systems.

This paper challenges some of the accepted wisdom as to the utility of RAD. Our study of the application of a RAD-like method within a large and complex development environment suggests that a more sophisticated relationship exists between an ISDM and its application. Our evidence leads us to suggest that the process of ISDM adoption and application is a dynamic one in which both the ISDM itself and the development environment are subject to mutual influence and change.

The structure of the paper is as follows. First, we review the characteristics of RAD as an ISDM and consider its credentials as an agile approach. Second, we consider the concept of a development environment and the critical characteristics that are generally deemed to influence the utility of the application of RAD. Third, we consider the methodology adopted in our research. Fourth, we present some of the results from our case investigation organized in terms of the characteristics of the particular development environment studied. Finally, we use our investigation to raise issue with what we see as a static conception of ISDM utility. In contrast, we argue for a conception of ISDM adoption and application as a dynamic and continuous process. We speculate that this lends support to the suggestions of Kautz *et al.* who argue that 'the numerous changes in information technologies and applications have not significantly changed the underlying persistent characteristics of ISD' (Kautz *et al.*, 2007, p. 218). ISD in practice continually consists of the 'unbundling' of context-independent *solutions* to meet the context-dependent *challenges* of a particular development environment.

RAD AS AN ISDM

ISDMs specify approaches for developing an IS within their wider organizational context. Within a large body of previous work, there are still limited practice descriptions and case studies of the selection, adoption and adaptation of ISDMs (Beynon-Davies & Williams, 2003). In this section, we consider the concept of an ISDM and the distinctive features of RAD as a particular instance of an ISDM.

Veryard (1987) equates an ISDM to an IS itself, and describes the implementation of an ISDM as an instance of both technological as well as an organizational innovation. Avison & Fitzgerald (2006) suggest that an ISDM typically encompasses a model of the development process, a set of development techniques, a documentation method, a perception of how these

would fit into the development process and a set of assumptions about what constitutes information, an IS and the place of IS within organizations.

Extant literature suggests that ISDMs are rarely adopted as specified within development settings. Organizations often either do not use any ISDM in practice, or adapt and use existing ISDMs to meet their specific requirements (Button & Sharrock, 1993). The suggestion is that practitioners adapt development practices to respond to the exigencies and situational requirements of the project and organization within which they find themselves.

There is some debate as to whether agile development approaches form ISDMs in terms of traditional conceptions as described above. For instance, agile 'methods' are generally contingent in the sense that they do not prescribe in detail development techniques to be used. A variety of techniques can be adapted to the needs of a particular project. A number of development methods are seen to sit under the umbrella of 'agile' development such as SCRUM (Schwaber & Beedle, 2002), Extreme Programming (Avison & Fitzgerald, 2006) and Dynamic System Development Methodology (DSDM Consortium, 2001). Some consider these approaches as 'the latest in the software field to be posited as the "silver bullet" to help address key problems in software development' (Fitzgerald *et al.*, 2006). The Agile Alliance (2001) in its 'Manifesto for Agile Development' places a high value on human roles and relationships rather than on tools and processes (Cockburn, 2002; Highsmith, 2002). Agile development emphasizes iterative and incremental development, intensive stakeholder involvement and joint collaboration and cooperation between stakeholders and developers in teams where responsibility is shared. All such properties are focused on the overall goal of achieving faster development (Cockburn, 2002; Avison & Fitzgerald, 2006).

RAD, first formalized by James Martin (1991), originated during the early 1990s from rapid prototyping approaches. RAD conforms to many of the principles of the Agile Manifesto. It uses an iterative model of the development life cycle in which software is produced using high-level development tools. RAD projects are managed in terms of fixed phases of development based around some form of prototyping with consequent heavy user involvement. Created in 1994, the DSDM is a prominent example of a RAD method and is an attempt to promote a 'best practice', non-proprietary public domain version of RAD (DSDM Consortium, 2001; Cockburn, 2002).

Iterative development refers both to the use of incremental prototyping and the incremental delivery of system elements. RAD is an approach that compresses the analysis, design, build and test phases of the development life cycle into short iterations ending in the demonstration of a deliverable to a relevant group of organizational stakeholders such as potential end-users of the intended system. A deliverable is thus a part of an information and communication technology (ICT) system agreed to be demonstrated at a review session by the development team to representatives of relevant stakeholder groups. A deliverable is normally expressed in terms of what the ICT system module will be able to do – its key functionality. RAD focuses on the rapid development of system functionality through the development of incremental prototypes evaluated in early user feedback. Thus, the system is built in an iterative way, utilizing a cycle of inspection–discussion–amendment until the user is satisfied with system.

Incremental delivery refers to the process of breaking down a project into development modules or units, each of which can be delivered separately as parts of a complete system. In this process, feedback from evaluations of initial modules feeds through into the design of subsequent increments developed.

RAD development typically takes place using joint teams of technical staff and organizational stakeholders, particularly potential end-users (Martin, 1991; Highsmith, 2002).

User involvement in system development is generally seen to influence the success of development projects (Hirschheim, 1983; Baronas & Louis, 1988; Kappelman & Maclean, 1991; Lynch & Gregor, 2004). Intensive user involvement is seen to cut across organizational barriers, reducing the effects of 'politics', and encourages timely decision-making leading to higher quality systems and greater business value from systems (Martin, 1991; Beynon-Davies *et al.*, 1999; DSDM Consortium, 2001).

Where it is not practicable to involve all potential end-users, the issue of identifying and involving the 'right' users becomes critical to the success of development (Hirschheim, 1983). The degree of decision-making devolved to users and their levels of commitment and availability to a project all affect the trajectory of iterative development (Beynon-Davies *et al.*, 1999).

Joint Application Design (JAD) is a systems development technique originated by IBM in the early 1970s, and further developed by James Martin (Elliott, 1997). Within RAD, joint design workshops form a fundamental part of iterative development. Typically, JAD sessions consist of teams of 4–8 key people, integrating developers and stakeholders empowered to make key decisions within the short iterations characteristic of this development approach (Martin, 1991; King & Galliers, 1994; Elliott, 1997).

Product-based project management associates the concept of a deliverable with that of a so-called timebox. A timebox is a scheduled period of work assigned to the development of some deliverable and is normally expressed in terms of a fixed, immovable deadline. The ideal is that a timebox is never changed once established. However, the functionality of a deliverable may be renegotiated to fit the timebox.

Product-based project management is also usually associated with the use of so-called clean rooms. JAD sessions take place in settings away from business and developer environments. The idea is that the workshops are free of interruptions from the everyday work environment such as telephone calls, messages, etc. Within such settings, techniques for change control such as MoSCoW prioritization are used for setting the scope of deliverables (Martin, 1991; Greer & Ruhe, 2003).

RAD development typically requires use of high-level development tools that automate aspects of the development process (King & Galliers, 1994). Use of high-level tools enables the production of automated documentation of system functionality. This allows streamlining of certain development activities and the emphasis to be placed within development upon the production of deliverables. A distinction is sometimes drawn between front-end tools supporting requirements analysis and design and back-end tools associated with code generation and implementation activities (Vonk, 1990).

Within our case study, high-level tools for the construction of business rules were critical to the development effort. A business rule encapsulates a component of business logic essential

to the performance of some given business process. Each business rule contains an action that gets performed when a condition is met. Tools are used to compile and store business rules in a database that can be loaded dynamically during run-time.

APPLICABILITY OF RAD

Previous research suggests that RAD is considered suitable for certain types of development environment. By development environment, we mean the organizational and project setting within which the development takes place. Certain key characteristics of such environments are seen as conducive to the application of RAD. In this section, we discuss three such characteristics particularly relevant to the focus of our case study: system functionality, project size and the profile of user involvement.

RAD is perceived as most suited to the development of systems with low levels of back-end complexity and high levels of front-end interactivity (Martin, 1991; Linthicum, 1997; Beynon-Davies *et al.*, 1999; 2000).

Most RAD projects build applications that are not complex computationally. The tendency is to rule out the applicability of RAD for large-scale, infrastructure projects such as corporate-wide database systems. Indeed, some practitioners propose that RAD projects can only be initiated once such infrastructure systems have been put in place. Hence, there is some consensus that RAD is not appropriate for real-time or safety critical systems, very large infrastructure systems or where functional requirements have to be fully specified before any program is written (DSDM Consortium, 2001). A further view is that the application of RAD is not appropriate where the application must inter-operate with existing programs, where it is not possible to utilize high-level development tools, or where extensive distribution of processes and data is concerned (Linthicum, 1997).

Most RAD projects are also conducted on applications that are highly interactive and have a clearly defined user group. The approach presupposes the use of prototypes typically of user interfaces and expects that detailed evaluations will be made of such prototypes through collaborative stakeholder involvement (Martin, 1991; Linthicum, 1997; DSDM Consortium, 2001).

The successful application of RAD is frequently associated with the issue of project size. RAD projects are generally characterized by small development teams – typically four to eight and a maximum of 10 people. This means that extant research suggests that RAD is generally considered inappropriate for large-scale projects (Martin, 1991; Beynon-Davies *et al.*, 2000). However, the breakdown of complex projects into smaller, more manageable components is proposed by some as a way forward for large projects, particularly those considered as high risk (Martin, 1991; McConnell, 1996).

Stakeholder involvement is considered an essential element of RAD work. However, such involvement has to be of the right form to facilitate the rapid development work characteristic of this ISDM. Three aspects of such involvement are seen as critical: the form of decision-making empowered to stakeholders involved, the degree of user and management

commitment to involvement and the form of communication between organizational stakeholders and developers.

RAD relies heavily on the ability of the stakeholders involved to make empowered decisions in a timely fashion without the need for further organizational approval. Where there is a culture of hierarchical decision-making in which subordinates are used to deferring decisions up the management line, this can seriously undermine the effectiveness of joint development. Morgan (1997, p. 169) argues that decision-making is inhibited by hierarchical cultures; he states, 'the limits of empowerment are usually quickly felt as people run into the constraints imposed by the existing hierarchy'. The nature of decision-making is hence culturally rooted and is a reflection of organizational culture (Schneider & Barsoux, 1997).

A RAD development approach depends not only on the capabilities of development professionals but also necessitates a high level of user and management commitment in terms of participation within the development arena and acceptance of the developing system. Within any pool of potential users, there will be different levels of knowledge and varying degrees of commitment relative to each projects' circumstances (Martin, 1991; Boehm, 1999; Beynon-Davies *et al.*, 2000).

The commitment of both managers to a RAD development and users to involvement in the project is deemed critical. The assignment of users to joint design teams is no guarantee of enthusiasm for such involvement. This problem of commitment relates to the frequently experienced difficulty of getting users to invest time and effort in a development project.

Effective communication within joint development is clearly a major component of successful project development within the RAD tradition. Generally, such communication is required to be open and transparent. Hence, great emphasis is placed within RAD on techniques for breaking down 'communication barriers'. Typically, within JAD, developers use tools and modelling techniques to communicate and confirm to users their understanding of the requirements communicated (Martin, 1991; King & Galliers, 1994; Elliott, 1997). Additionally, use of prototyping provides stakeholders with a tangible means of understanding and evaluating the system being developed, as well as presenting developers with more meaningful feedback, and introducing a baseline for progressing development (Alavi, 1984; Vonk, 1990).

RESEARCH METHOD

The researchers adopted an interpretive stance (Walsham, 1997) within an organizational setting using an ethnographic method of qualitative data collection. This approach was selected because the researchers wanted to gain a broad understanding of the application of a particular ISDM within its organizational context (Gill & Johnson, 1991; Walsham, 1997). Such an interpretive epistemology has been adopted in a number of studies within the IS domain (Orlikowski & Baroudi, 1991; Beynon-Davies, 1997; Myers, 1997).

Our study consisted of longitudinal research over 3 years, situated within the project setting. It involved a sustained period of fieldwork by one of the researchers enabling close observation of people and events as they occurred in practice in their natural context (Silverman, 2005).

Entry into the project arena was established through the project board of the development project. Access was granted across the project setting and to all participants. With reference to Adler's typology, the fieldworker's role represented a peripheral presence within the project arena that was not an active input into its central activities (Adler & Adler, 1987 cited in De Laine, 2000). Qualitative data collection consisted of observations, documentary analysis and a series of in-depth interviews with project participants. This combination of approaches enabled the study to achieve a 'thick description' of the situation and its context.

The initial 9 months of the research study consisted of continual non-participatory observation of project work on a daily basis. This particularly consisted of attendance at both formal and informal meetings relating to project work. Meetings attended varied between daily update meetings, biweekly progress meetings and weekly round-table meetings. Attendance also occurred at requirements gathering workshops, design/development briefings, stakeholder information seminars and discussion groups. Extensive field notes were recorded regularly in a project diary to facilitate the auditing and tracking of data/events (Myers, 1999). Where concurrent activities prevented direct observation, feedback was sought from participants through indirect observations (Gill & Johnson, 1991).

Key participants were shadowed for 1 or 2 days to obtain specific knowledge and insight relevant to the project focus at that time. In this way, the researcher was able to gain partial understanding and explanation of what had been observed but not necessarily understood (Patton, 1990). This exercise also enabled the later identification of people as key informants to be interviewed.

A number of project documents were collected as part of the investigation. This documentation ranged from technical specifications such as the logical data model used for the system through to method detail such as the process to be followed for the development of business rules. This documentation particularly enabled greater understanding of aspects of the planned systems design and development process.

Informal semi-structured interviews were conducted across the project population for the duration of the study. One hundred twenty-six one-to-one interviews of approximately 1 hour's duration were undertaken, which sustained anonymity and confidentiality. Informed consent was central to the interviews such that all interviewees understood the nature of the research being undertaken, its aim and the anticipated outcomes (Silverman, 2005). Interviews were audiotaped, transcribed within 2 days and validated with respondents for added rigour and to offset unintentional observer bias (Patton, 1990). The aim was to maintain an ethical stance to afford the interviewee the opportunity to clarify, delete or amend data that were either sensitive, inaccurate or which had inadvertently 'slipped out' during the interview sessions, thus providing valuable feedback

CASE STUDY

Within this section, we describe how a commercial ISDM (a RAD-like approach) was adopted and applied within a particular organizational setting. In previous work, we have found it useful

to consider a particular ISDM as a decontextualized knowledge bundle that undergoes a process of contextualization in its application within a particular organizational setting (Beynon-Davies & Williams, 2003). As we shall see, a given ISDM was imported into the case organization as part of a major outsourced development project. Elements of this ISDM were unbundled and contextualized over a number of years within this organization. This unbundling process, by its very nature, consists of interplay between the shape of the ISDM itself as a 'knowledge bundle' and key facets of the organization such as its structure and culture. Much of this interplay initially proved problematic in the sense that facets of the ISDM clashed with existing aspects of organizational structure and culture. However, over time there is evidence of a successful diffusion and infusion of certain ISDM practices within the case organization.

Under recent UK Government devolution legislation, a UK regional government department (the clients) took on the devolved functions for administering all aspects of the European Union's (EU) Common Agricultural Policy (CAP) schemes across the region. As an accredited EU paying agency, the department has a legal obligation for administering the European Agricultural Guidance and Guarantee Fund grants and subsidy payments to customers across the region through a number of CAP schemes. It is responsible for making these payments within a specified time frame and must ensure that all EU regulations and compliance issues have been met before payment is made.

Organizational setting

The CAP Management (CAPM) Division is the operational side of the regional government department that deals with the administration of CAP schemes. It is run from a number of locations throughout the region, and consists of approximately 540 staff dealing with many thousands of scheme payments totalling in excess of £200 million annually. The headquarters of CAPM is located centrally and the organization also includes three divisional offices (DOs) and seven area offices spread across the region. These act as the primary source of management of CAPM functions.

The case study organization is best described as bureaucratic in that it is strongly hierarchical, highly procedural and risk averse. Work is conducted in a highly regulated environment within a culture which interviewees described as a 'blame culture' (Morgan, 1986; Hofstede, 2003). The organizational structure consisted of a hierarchy with clear management lines of responsibility and authority supporting work processes that are highly organized, compartmentalized and systematized. Individual work is highly specialized and domain knowledge is seen to belong to particular individuals.

CAPM is responsible for managing the expenditure of CAP grants and subsidies designed to support producers in the farming community who meet the specified scheme criteria across different schemes. Each of the schemes, some of which are payable more than once a year, operates within an annual cycle that coincides with a particular fixed control period set by the EU. These are aligned to a 'calendar window framework' and some schemes operate year-round, enabling multiple claims to be made. In order to claim for a CAP grant or subsidy, a

scheme application claim form must be completed and submitted by applicants within the specified time 'window' of the desired schemes' life cycle.

The CAP schemes do not exist independently of each other but form a network of complex, interdependent relationships. Not only are they computationally complex; they are also individually dependent on specific payment windows within the annual cycle. Associated payments must conform to the EU legislation and EU regulatory control mechanisms that undergo continual change.

There are two EU compulsory control mechanisms. First, the Customer Details System, a former EU directive that specifies that all member states must have a common integrated system to handle all communications with its customers. Second, an Integrated Administration and Control System (IACS) that defines the common rules for the administration and control of EU support schemes. This control mechanism encompasses data that cross-references across a number of CAP schemes with the purpose of combating fraud. All potential grant and subsidy recipients must complete both an IACS form and a Customer Details Wales (CD(W)) form.

There are six principal CAP schemes including an Arable Area Payment Scheme, a Beef Special Premium Scheme, an Extensification Premium Scheme, a Sheep Annual Premium Scheme, a Suckler Cow Premium Scheme and a scheme for mountain farming – Tir Mynydd. Each scheme has a number of related sub-schemes. Figure 1 represents a simplified, diagrammatic view of relationships between schemes and control mechanisms. This is intended merely to illustrate the complexity of relationships that formed the design space for the new IT system described below.

The dynamic nature of the EU's agricultural policy means that schemes are frequently changed with new schemes being drafted by the member states as required. Each year, the EU issues a number of directives that impact on the administration and management of the CAP schemes that all member states must comply with. Thus, as a part of the UK, the Regional Department is answerable directly to the EU and at risk of disallowance, i.e. fines and penalties being imposed by the EU for irregularities in CAP administration.

Project initiation

Historically CAP scheme administration was organized into separate silos where people worked together in small, discrete but independent teams to deal with specific schemes and their requirements. Scheme management was the responsibility of scheme-specific process and scheme managers. It was these managers who attended to the business needs and administration of schemes. We refer to them collectively as business managers.

Prior to the initiation of the project, which is the subject of our case study, CAPM performance was seen to be of an inadequate and poor standard. There was a history of late payments, poor customer satisfaction and an increasing inability to meet the EC's changing requirements. Historically, a separate IT department within the organization dealt with the maintenance of the existing legacy system supporting CAP scheme work. This IT department consisted of 120 operations support staff, and extended across the three DOs. However, they

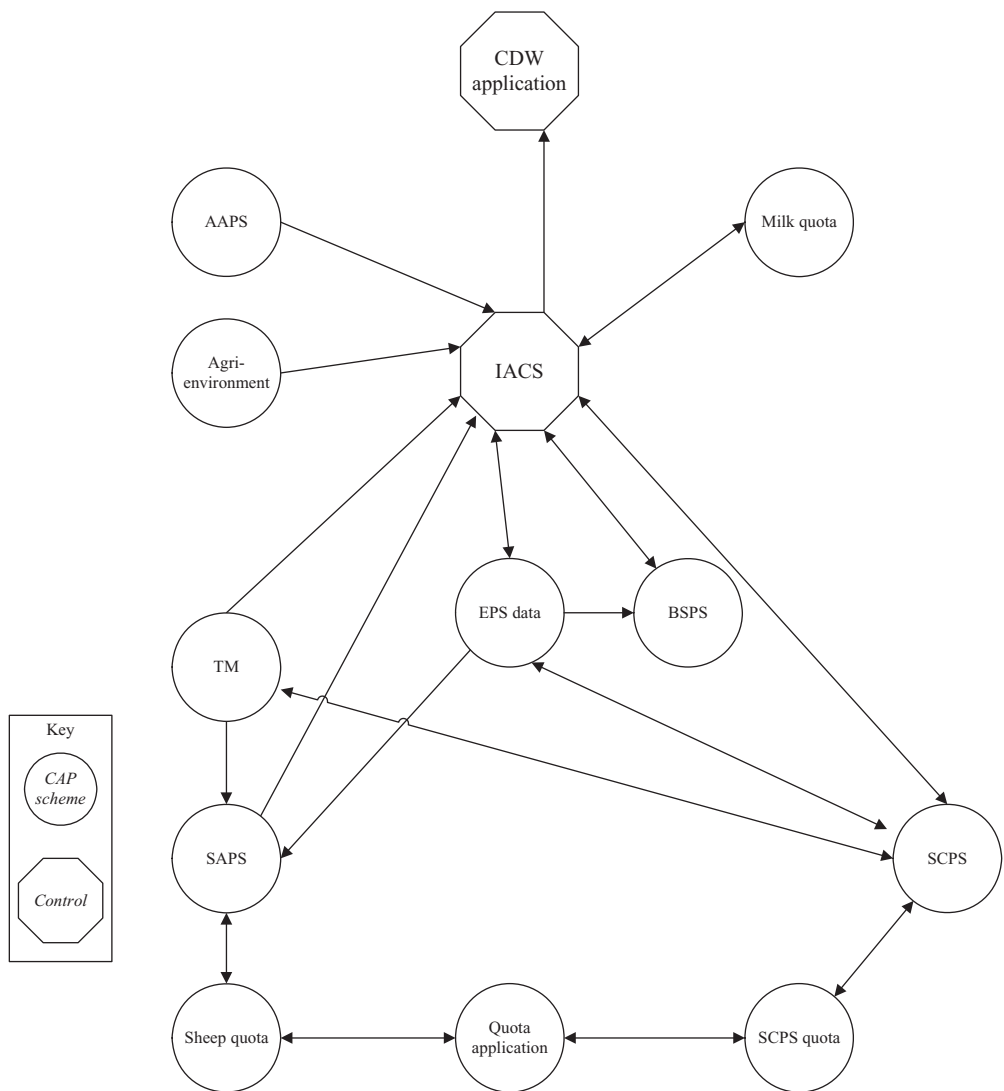


Figure 1. Relationship between Common Agricultural Policy (CAP) schemes. CDW, Customer Details Wales; AAPS, Arable Area Payment Scheme; IACS, Integrated Administration and Control System; EPS, Extensification Premium Scheme; BSPS, Beef Special Premium Scheme; TM, Tir Mynydd; SAPS, Sheep Annual Premium Scheme; SCPS, Suckler Cow Premium Scheme.

had a perceived reputation within the organization for delivering development work late and that did not always meet user needs. An internal evaluation of the service and the legacy system revealed poor customer satisfaction and an increasing inability to meet the EC's changing requirements. One business manager reported that the need for a new system was

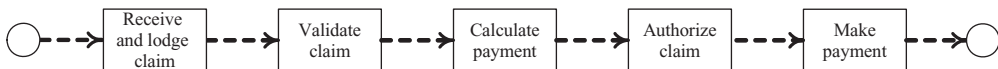


Figure 2. Generic Process Model.

long overdue and described the legacy system as 'somewhat antiquated and outdated . . . it had had so many little add-ons, bolt-ons and new programs added to it, it must have been creaking at the seams' [Business Manager 4]. Another reflected, 'the Agriculture Division was performing very badly. We were being out performed by the other areas within the UK; payments were delayed and sometimes inaccurate. Basically in every area we were performing badly [Business Manager 16]. Such problems provided the justification for the development of a new IT system. A business case was produced, considered and accepted by senior members of the regional administration. The case for the new IT system was intended to move away from the previous individual scheme administration procedures towards a Generic Process Model which integrated the core processes of separate CAP schemes. The business case specified the need for an integrated set of re-designed and standardized processes that would facilitate automated data capture, data validation and speedier payments to customers.

The project to deliver the new IT system was large in a number of senses. Its size and complexity are reflected by an initial cost estimate of more than £10 million, a projected time frame of 2–3 years and a core project team of more than 50. It is not possible to measure the number of customers precisely as 'customer farmers' apply for any number of scheme grants and subsidies relevant to their individual situations. Some of the schemes are payable more than once a year, while others operate year-round enabling multiple claims to be made. However, the extent of the customer base is traditionally measured in terms of total payout of EC monies – totalling approximately £200 million annually spread across the UK region.

In accordance with EU Legislation, an invitation to tender was placed in the *Official Journal of European Communities*, resulting in 76 expressions of interest from external suppliers. From a final shortlist of three, a preferred supplier was identified and placed before the project board and permanent secretary for approval. The developers (as we shall refer to them) were selected on the basis of their experience in the field of systems transformation and of developing customized software and hardware. The developers entered into a contractual agreement to work in partnership with the client organization to design, develop and implement the new IT system. All development work on this particular project was therefore outsourced to the developers.

The new IT system was aimed at improving the effectiveness and efficiency of CAP grants and subsidy administration. It moves away from the former discrete individual scheme administration processes towards a generic process (Figure 2) that integrates the core processing of the common activities across separate schemes.

ISDM description

The developers proposed use of their own in-house commercial RAD-like development approach known as Iterative Application Development (IAD). Examples of previous successful

development applications across the globe were put forward by the developers as evidence of the suitability of the ISDM. These included facilitating a modular approach involving legacy systems for a European market-leading electrical company in Sweden, a development project for a central securities depository in Norway, and replacing a business critical system for one of the largest communications companies in the UK.

IAD adopts both notions of iterative development and incremental delivery. In a typical project in which IAD is applied, four key development stages are involved, consisting of time-boxed development iterations. In terms of the project under consideration, the developers envisaged completing the initial development work during stage 2 and then revising and modifying the system to incorporate new business needs during iterations in stages 3 and 4. However, the iterations did not occur as intended. Stage 1 was completed as planned but towards the end of stage 2, the EU imposed a directive that radically changed major requirements in the system.

This EC reform represented a fundamental change in the way CAP grant and subsidy payments are awarded. Grants and subsidies would be decoupled from production-based payments via the existing CAP schemes towards a single farm payment provision that represented a fairer distribution of EC monies. This would replace the current administration of schemes and radically alter the operation of the CAP schemes themselves. These changes impacted across the development and implementation of schemes being prepared for the new IT system. It also meant that a number of the schemes would cease to exist as individual schemes in the following year.

This level of change to requirements caused a major reassessment of the project. One business manager stated that, '... certainly I don't think that this Reform is common. I think what they are actually proposing to do now is quite extraordinary' [Business Manager 16]. Consequently, a critical assessment was made as to the viability of continuing existing development and of the proposed scheduled development planned for the new IT system in terms of investing time, effort and resources. A high-level decision was made to maintain the schemes, or parts of schemes, already using the generic process, but to halt new development of schemes waiting to go onto the generic process that would cease to exist. Thus, development on the case study project was halted and a revised project emerged to accommodate the new development trajectory. This work is still ongoing.

The application of IAD involved a sequence of short, time-boxed iterative development cycles that adhered to a 'fit for purpose' philosophy rather than building 100% of business needs. The developers were provided with a requirements catalogue and the proposed Generic Process Model concept at procurement and this formed the basis and scope of planned development activities.

Figure 3 illustrates how project development was broken down into development modules that were initialized with process definitions relevant to the development stages of the Generic Process Model. Each of these modules was intended to go through build, test and release stages before passing through integration testing, system testing and user acceptance testing. If testing proved successful, then a development module was released into live production. However, if it failed at any point, a test incident report was raised and issues dealt with in the next iteration of the development cycle.

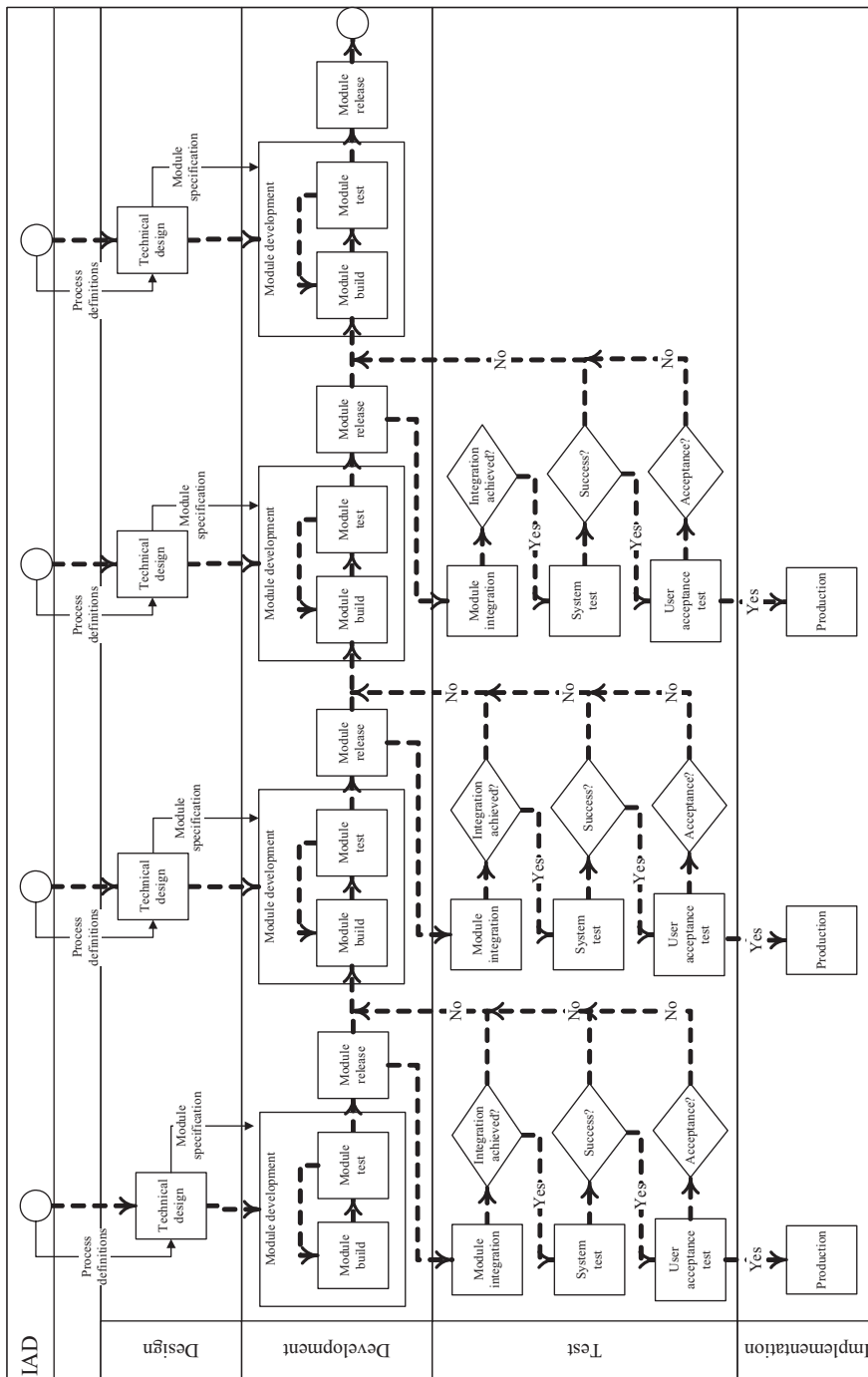


Figure 3. Iterative Application Development (IAD) development cycles.

JAD workshops were used to 'drill down' into the detail of requirements. This provided the functionality of the business processes as identified through the initial high-level requirements catalogue. A business rules engine was used to manage scheme-specific business rules aligned to the generic processes that accommodated the complexities of the CAP schemes.

Project environment

Although development was outsourced to the developers, technical staff was co-located at the same central location as the clients for the duration of the project. The project structure consisted of a senior management project board and teams of project workers with a pre-defined reporting structure. Teams were subject-specific according to need that evolved in line with the project's development.

The 'project population' was a term used to refer to the outsourced developers, the organizational people within the IT/IS development arena and members of the project board. It more specifically describes the business people selected by the senior management team of the client organization from the 'pool' of potential stakeholders (Beynon-Davies *et al.*, 1999) such as senior managers, business and process managers and other CAP business-related personnel. These people were identified as key knowledge holders, and were explicitly empowered with decision-making responsibility pertinent to the development of the new system. Hence, the term stakeholder community became associated specifically with a core team of more than 50, composed of both developers and organizational representatives.

Application of RAD

As mentioned above, a high level of stakeholder involvement characterized by commitment and availability is considered important to the successful application of the RAD approach. Coupled with this, user-developer communication needs to be open and transparent. In terms of our research study, the term 'user involvement' refers specifically to the involvement of members of the senior management team, business managers (process and scheme managers) and the developers responsible for the separate CAP schemes, within joint decision-making.

Within the first couple of years of the project, problems were experienced with such involvement, which impacted upon project trajectory. Towards the end of the project, some of these issues began to be addressed and a greater appreciation of the need for effective stakeholder involvement in projects of this nature appears to have infused throughout the organization.

Organizational stakeholders initially assigned to the project found it difficult to adjust to the requirements of JAD work. As a consequence, within JAD workshops, passive user involvement was the norm. Organizational stakeholders were generally unwilling to commit to fast and authoritative decision-making for a number of reasons. Where an organizational culture stresses individuality and individual accountability (i.e. a 'blame culture'), the concept of team

working is likely to be problematic. This clearly jarred with the joint development ethos and team accountability demanded of RAD work (Martin, 1991; McConnell, 1996).

Established organizational culture dictated that decision-making should be located at the appropriate point in the organizational hierarchy. This meant that business managers remained reluctant to make decisions on business processes that they felt responsible for despite being empowered to do so. This appears to be because of the blame that would be attached to any wrong decisions. The project manager (an employee of the client organization) confirms this stating: 'The ability to make effective decisions is a cultural aspect of the Department in that people don't particularly want to (I'm glad you are recording this actually) people don't want to make decisions because they don't want the blame attached to them if something goes wrong'. The developers echoed this. For example, one developer [Developer 21] indicated that users '... were empowered to make decisions but they just couldn't. It was very frustrating we were trying to meet deadlines but this, well it just makes it very difficult'.

Associated with this reluctance was an inability to prioritize the needs of the organization over and above the needs of specific scheme management. Developers commented, 'As far as the business managers were concerned it was only their scheme that they were interested in, so they did not promote the dependencies of the other schemes' [Developer 9]; '... actually what happened was everybody was still saying "my priority is first, mine's the first", from 5–6 different Business Leaders' [Developer 21].

Against this background, there is clear evidence of a lack of trust between developers and business managers, and this engendered a level of conflict. In some of the initial design workshops during stage 2 which involved the developers' business analysts and scheme-related business managers, conflict arose over the level of business detail necessary for the new system. At this stage, developers were concentrating on development of the Generic Process Model working to a fixed schedule of activities and were reluctant to collaborate with business managers outside these set parameters. In contrast, the business managers remained resolute in their belief that the Generic Process Model represented only a high-level overview of the new system. They were committed to a design that would meet all their business needs and which went far beyond the proposed model.

As a consequence, there was no common vision and therefore no joint team identity, unity or spirit; there was no 'we' and ultimately no trust necessary for the IAD approach. The following comments illustrate the frustration experienced by the business managers at the time: '... we would have the discussion with the Business Analyst (developers) and it was as if there was a predefined end product, in that you'd have a discussion and it would be 'you can't have that', and 'you can't have this, this is what we think you need [Business Manager 14]. 'I don't think XX was prepared to listen enough to business experts [Business Manager 2], '... what we were saying wasn't particularly taken on board and what we were asking for either you can't do that or, there was a lot of 'that's out of scope' [Business Manager 2], and 'It's created a lot of problems, a lot of unease and a lot of distrust now' [Business Manager 4].

The developers were equally troubled. For example, '... for the first 8/9 months I spent most of my time telling the Business "sorry that's not in my scope", because I had a very strict list of requirements that I had to try and deliver ... [Developer 6: 1st interview], 'I know for a long

time that I kept having to tell the Business “no that’s not in scope” which frustrated them enormously. [Developer 6: 2nd interview]. Thus an environment of trust necessary for the IAD approach was compromised – ‘The trust has broken down; we anticipated a trust partnership but that has not happened’ [Developer 19].

Discourse not surprisingly started to reflect a perception of two sides, ‘us’ and ‘them’. For example, ‘We had a lot of difficulties, there was definitely an “us and them” attitude’ [Developer 8]. This was confirmed by business people – ‘There was this divide, it was very “them and us”’ [Business Manager 5].

This reluctance to be fully involved and the conflict it engendered impacted upon the development in a number of ways.

For example, conflict led to difficulties in specifying requirements, which led in turn to a larger amount of development iteration and testing. One manager stated, ‘we underestimated the amount of testing that was required, we underestimated the number of iterations that application development was going to have to go through and the estimates I was getting at the time, which all seemed perfectly reasonable, were not being realised in practice’ [Senior manager 1]. This led to delays in module releases: ‘I remember, XX was going to be released this afternoon, it was 6 weeks before it was released yet it was supposed to be that day . . .’ [Business Manager 3]; ‘YY lacked some elements – developers promised them within 3–4 weeks, it actually took 3–4 months because it was so complicated’ [Business Manager 7]. This caused problems with overall scheme administration, such as when a particular CAP scheme’s payments scheduled for a particular year were delayed by 9 weeks because increments of the new IT system were not operational in time to meet October payment deadlines. As a consequence, this piece of development was halted and the scheme was reassigned back onto the legacy system to enable payments to be made within EC deadlines.

Business managers reflected retrospectively that these difficulties could have been controlled through stronger and more direct supervision. For example, Business manager 4 comments, ‘I believe that there is a lack of visibility in the project management towards the business managers’. Business manager 6 confirmed this: ‘I think the impact on the project was it could have used greater leadership skills’, and another said, ‘Project management was weak’ [Business Manager 17].

Hence, there was a perception that the authority exerted at the time was not sufficient or effective enough to handle the impact that these issues had on initial development delays. For example, a senior developer said ‘Every Process Manager was saying “mine is as important as yours, and my dates are just as important as yours”. It needed somebody at a level above them to say right you Process Managers actually yours is less important than that one. I want that one delivered and I want it delivered on time’ [Developer 1].

The inability to make empowered decisions about business needs was a key concern for the developers who needed prioritization of development work to meet time-boxed development deadlines. As a response, senior management created the roles of business champions, which were to be responsible for key business areas/functions and empowered to make decisions in these areas. The objective was to engender motivation, and encourage movement from a risk averse culture to a more risk-taking environment. However, the subtle nuances embodied in

these changes were not effectively communicated to those involved. Hence, there was evidence of continued risk aversion. For example, Business manager 6 commented 'I was a Business Champion . . . we were just puppets really, we were Champions so they could blame somebody when things went wrong'.

Channels of communication were also markedly restricted in the sense that they were inherently vertical rather than horizontal. This did not facilitate easy diffusion of critical business decisions. One member of the project team reflected this in the comment, ' . . . so you'd have different meetings with all of them and then you'd say "we told 'so and so' (developer) that the other day" and they'd say "oh he hasn't told us that" ' [Business Manager 3].

Developers commented, 'I'm not quite sure who was responsible for communicating what . . . I always got the impression that they were one-way communication exercises, I didn't hear much coming back to the IT stream' [Developer 11]; ' . . . the external communication wasn't as good as it could have been but definitely, absolutely the internal communication was terrible' [Developer 8]. The business managers agreed 'No, initially there was no contact at all with developers or the business analysts, or very little' [Business Manager 14]. This affected the project in terms of reduced motivation and commitment.

Unbundling of an ISDM

Despite the problems described above, over time, we observed a gradual acceptance of the promoted ISDM practices within the client organization. In the early stages of the project, the IAD method was only partly embraced by project participants. The developers, senior managers and, to a certain extent, business personnel were receptive to adopting the practices of the ISDM. However, in its early stages, attitudes and behaviours characteristic of the inherent working ethos and day-to-day practices of the organization initially conflicted with those required for effective application of the RAD-like approach. These findings concur with Jones & King (1998) who found that RAD as an ISDM is problematic for highly centralized and bureaucratic organizations. The inherent hierarchical nature of communication and decision-making in such organizations runs counter to the fast, iterative nature of RAD development that emphasizes speed of authoritative communication and involvement of key business decision-makers throughout.

However, the practices within any development project, particularly of the scale of the one described, are likely to evolve in terms of their interaction with the relevant organization. There is evidence in our case of an evolving rate of adoption of aspects of the IAD method which initially proved problematic. As the project matured and development evolved through the subsequent second project environment (year 3 onwards), it is clear that business managers began to accept some of the initial, problematic elements. for instance, 'What I've noticed is that although their work is very important they're now accepting that there are items that other process managers have that are more important from the total CAPM business perspective. I think it's evolved . . . It is now really "Smart", the way we are working is smart now' [Senior manager 12]. As familiarization with the approach developed, interviewees reported a greater acceptance of the need for collaboration and effective decision-making.

Indeed, there are high expectations of success with the subsequent version 2 of the project that is continuing to evolve. Business managers were beginning to say 'OK I understand I have a priority but I also appreciate that yours is more actually more important . . . and I'll look at compromise solutions for mine' [Business Manager 5]. Senior manager 1 reflects, 'I think over the last 12 months it has become more effective but as usual you learn as you are going along'.

Interestingly, in interviews with clients and developers at the time of transition between the two major phases of the project, expressions were voiced that, although the journey had been difficult, the application of the ISDM approach was seen as successful, particularly because it enabled the project organization to deal with an evolving and volatile set of requirements. Thus, the RAD-like approach adopted was particularly recognized as important in handling the uncertainty experienced within the development arena. A senior manager reflecting on joint team development commented, 'It's got better and better, but that's all about people and the way that they work together and the fact that they can see that both sides . . . sharing some of the pain, sharing the objectives, sharing in the success . . . that all inculcates a really good spirit of team working and I think probably tend to try and bring those two cultures together' [Senior manager 10]. He concluded, 'one lesson that I suppose I would take out of this is a positive one which is that managing a project on this basis, which is the creation of a joint team, has been excellent and I'm really very, very pleased with the way that's worked and indeed we will do that in future'.

Therefore, our interviews with both developers and the client organization clearly establish the belief among both groups that had a traditional linear development approach been adopted, the project would have been cancelled during its second year. It would also have not proceeded into the second project that evolved out of the initial development work. We can hence speculate that the major change in requirements caused by the EC reform described above proved a critical point in the project. One business manager stated, 'If CAP Reform hadn't come along . . . we would have had to asked for an extension' [Business Manager 7]. At this point, ISDM practices had started to infuse among the clients and the need to start afresh stimulated a new level of commitment on the part of the organization. Business manager 7 remarked 'Yes, it's breathed some life into it'. Also, there was an increasing accommodation on the part of developers to the complex nature of the system to be developed.

CONCLUSION

In this paper, we have considered the application of a RAD-like approach within a development environment characterized by properties which previous research predicts would militate against its successful adoption. RAD, like most agile development approaches, is generally proposed for projects in which the end product is generally of low background complexity but with high front-end interactivity. RAD projects are also typically of relatively short duration and undertaken by small joint development teams, in which decision-making is empowered and consequently speedy.

The development environment within the studied case consisted of a project of large duration and with many people involved. It focused on the construction of a system with considerable back-end complexity and with little front-end interactivity. The project also took place within a bureaucratic culture subject to hierarchical decision-making, vertical communication and risk aversion. The development environment was therefore not immediately conducive to elements of a RAD-like ISDM.

Not surprisingly, our case study demonstrates that a number of problems were experienced with the application of iterative development principles, particularly surrounding the conduct of stakeholder involvement within joint design. In general, stakeholders were initially passive in their involvement and, although formally 'empowered', proved reluctant to make decisions outside of their expected positions. Communication within design sessions was also restricted rather than open. Such difficulties impacted upon project trajectory causing delays in meeting key project deadlines.

However, our case study demonstrates that the adoption of an ISDM is not necessarily a linear, all-or-nothing process. A gradual unbundling and situational contextualization of a particular ISDM within its organizational context can be expected over time (Beynon-Davies & Williams, 2003). Indeed, our case study suggests that there are perceived benefits to being large in an outsourced development relationship, at least as far as ISDM diffusion is concerned. We might speculate that within smaller development environments, there is little scope for outsourced developers to engage with the organization in the active promotion of the practices of their chosen ISDM. This may partly explain some of the difficulties experienced in RAD or agile practices more generally. However, in a large-scale, long-term project, there is scope both for a wider range of stakeholder involvement and greater time for practices to diffuse and infuse within the holding organization.

Our involvement in this study, therefore, leads us to speculate that ISDM adoption and application is not a static process with a single decision point for assessing the utility of an ISDM. Instead, ISDM adoption and application must be considered a dynamic and continuous process that affects not only the ISDM itself but also the organization within which it is applied. The problems experienced in our case study were undoubtedly due in part to established organizational culture. Organizational members brought previous organizational expectations and work practices into the development work. However, our evidence suggests that such problems appear to have lessened over time due to the evolution and familiarization of the development approach within the organization.

Kautz *et al.* argue that 'the numerous changes in information technologies and applications have not significantly changed the underlying persistent characteristics of ISD' (Kautz *et al.*, 2007, p. 218). They argue that contemporary forms of ISD such as agile approaches are an accentuated evolution, rather than revolution, of well-known challenges and solutions. Challenges comprise: the increasing diversity of IS projects in terms of size, application domain, underlying technology and the number of people involved in and affected by such systems; the dependence of successful development on developer's knowledge of technologies, applications and people and the uncertain relationship between structure (organizational, project and method) and development practice. Practices for coping with these problems comprise

absorbing and reducing diversity through the application of organization and specialization, constant verbal communication and negotiation and pragmatic application of certain development methods and methodical concepts.

To a certain extent, our case material supports aspects of this analysis. Feedback from project participants within our case study suggests that certain challenges evident in the development environment such as the uncertainty associated with EU requirements began over time to outweigh certain structural problems associated with issues of bureaucratic organizational culture. For instance, although participation within joint development teams proved initially problematic, both developers and organizational stakeholders appear to have eventually been forced to engage in greater communication and negotiation. Also, certain methodical concepts characteristic within RAD as an ISDM such as empowered decision-making among joint design teams are now acknowledged as critical to this process.

These findings have clear implications for practice. On the one hand, our case study supports previous research in establishing that the application of agile ISDMs, in general, and agile techniques, in particular, within a large, complex and rigid development environment is likely to prove problematic. On the other hand, our study extends previous research in suggesting that initial problems with such application can be overcome both in the mutual process of ISDM adaptation and organizational change.

ACKNOWLEDGEMENTS

The researchers extend their gratitude to all participants involved in the data collection activities of both the developer organization and the government department for their cooperation and contribution during this research study. We would also like to thank the National Assembly of Wales for funding this research.

REFERENCES

- Alavi, M. (1984) An assessment of prototyping approaches to information systems development. *Communications of the ACM*, **27**, 556–563.
- Avison, D.E. & Fitzgerald, G. (2006) *Information Systems Development: Methodologies, Techniques and Tools*, 4th edn. McGraw-Hill, London, UK.
- Baronas, A.K. & Louis, M.R. (1988) Restoring a sense of control during implementation: how user involvement leads to system implementation. *MIS Quarterly*, **12**, 111–124.
- Baskerville, R. & Pries-Heje, J. (2004) Short cycle time systems development. *Information Systems Journal*, **14**, 237–264.
- Baskerville, R. Pries-Heje, J. & Ramesh, B. (2007) The enduring contradictions of new software development approaches: a response to 'Persistent Problems and Practices in ISD'. *Information Systems Journal*, **17**, 241–245.
- Beynon-Davies, P. (1997) Ethnography and information systems development: ethnography of, for and within IS development. *Information and Software Technology*, **39**, 531–540.
- Beynon-Davies, P. & Williams, M.D. (2003) The diffusion of information systems development methods. *Journal of Strategic Information Systems*, **12**, 29–46.

- Beynon-Davies, P., Carne, C., Mackay, H. & Tudhope, D. (1999) Rapid application development (RAD): an empirical review. *European Journal of Information Systems*, **8**, 211–223.
- Beynon-Davies, P., Mackay, H. & Tudhope, D. (2000) It's lots of bits of paper and ticks and post-it notes and . . . a case study of a RAD project. *Information Systems Journal*, **10**, 195–216.
- Boehm, B. (1999) Making RAD work for your project. *IEEE Computer*, **32**, 113–117.
- Button, G. & Sharrock, W. (1993) The discourse of negotiation. In: *Practices in the Work of Ordering Software Development*, Firth, A. (ed.), pp. 159–180. Pergamon, Oxford, UK.
- Cockburn, A. (2002) *Agile Software Development*. Addison Wesley, Boston, MA, USA.
- Coughlan, J. & Macredie, R.D. (2002) Effective communications in requirements elicitation: comparison of methodologies. *Requirements Engineering*, **7**, 47–60.
- De Laine, M. (2000) *Fieldwork, Participation and Practice – Ethics and Dilemmas in Qualitative Research*. Sage, London.
- DSDM Consortium (2001) Dynamic systems development method. [WWW document]. URL <http://www.dsdm.org/version4/2/public/> (accessed 14 February 2002).
- Elliott, E. (1997) Rapid applications development (RAD): an odyssey of information systems methods, tools and techniques. *4th Financial IS Conference*, Sheffield Hallam University, UK.
- Fitzgerald, B., Hartnett, G. & Conboy, K. (2006) Customising agile methods to software practices at Intel Shannon. *European Journal of Information Systems*, **15**, 200–213.
- Gill, J. & Johnson, P. (1991) *Research Methods for Managers*. Chapman, London.
- Greer, D. & Ruhe, G. (2003) Software release planning: evolutionary and iterative approach. *Information and Software Technology*, **46**, 243–253.
- Highsmith, J. (2002) *Agile Software Development Ecosystems*. Addison-Wesley, London.
- Hirschberg, M.A. (1998) Rapid application development (RAD): a brief overview. *Software Tech News*, **2**, 1–7.
- Hirschheim, R.A. (1983) Assessing participatory systems design: some conclusions from an exploratory study. *Information and Management*, **6**, 317–327.
- Hofstede, G. (2003) *Cultures and Organisations – Software of the Mind*. Profile Books, London.
- Jones, T. & King, S.F. (1998) Flexible systems for changing organisations: implementing RAD. *European Journal of Information Systems*, **7**, 61–73.
- Kappelman, L. & Maclean, E. (1991) The respective roles of user participation and user involvement in information systems implementation success. In *Proceedings of International Conference on Information Systems*, pp. 339–348, New York, NY, USA.
- Kautz, K., Madsen, S. & Norbjerg, J. (2007) Persistent problems and practices in information systems development. *Information Systems Journal*, **17**, 217–239.
- King, S. & Galliers, R. (1994) Modelling the CASE process: empirical issues and future directions. *Information and Software Technology*, **36**, 587–596.
- Linthicum, D.S. (1997) Driving development. DBMS (Online) 10. [WWW document]. URL <http://www.dbmsmag.com/9704d14.html> (accessed 26 April 2009).
- Lynch, T. & Gregor, S. (2004) User participation in decision support systems development: influencing system outcomes. *European Journal of Information Systems*, **13**, 286–301.
- McConnell, S. (1996) *Rapid Development – Taming Wild Software Schedules*. Microsoft Press, Redmond, WA, USA.
- Martin, J. (1991) *Rapid Application Development*. Macmillan, New York, NY, USA.
- Morgan, G. (1986) *Images of Organization*. Sage, London.
- Morgan, G. (1997) *Imaginization: New Mindsets for Seeing, Organizing and Managing*. Berrett-Koehler, San Francisco, CA, USA.
- Myers, M.D. (1997) Qualitative research in information systems. *MIS Quarterly*, **12**, 241–242.
- Myers, M.D. (1999) Investigating information systems with ethnographic research. *Communications of the AIS*, **2**, 1–20.
- Orlikowski, W.J. & Baroudi, J.J. (1991) Studying information technology in organizations. *The Institute of Management Sciences*, **2**, 1–28.
- Patton, M.Q. (1990) *Qualitative Evaluation and Research Methods*. Sage, London.
- Schneider, S. & Barsoux, J.L. (1997) *Managing Across Cultures*. Prentice Hall, Herefordshire, UK.
- Schwaber, K. & Beedle, M. (2002) *Agile Software Development with SCRUM*. Prentice Hall, Hemel Hempstead, UK.
- Silverman, D. (2005) *Doing Qualitative Research*. Sage, London.
- Truex, D.P., Baskerville, R. & Klein, H. (1999) Growing systems in emergent organizations. *Communications of the ACM*, **42**, 117–123.
- Veryard, R. (1987) Implementing a methodology. *Information and Software Technology*, **29**, 469–474.

Vonk, R. (1990) *Prototyping, The Effective Use of CASE Technology*. Prentice Hall, New York, NY, USA.

Walsham, G. (1997) *Interpreting Information Systems in Organizations*. John Wiley, New York, NY, USA.

Biographies

Hilary Berger is a Senior Lecturer in the Cardiff School of Management at University of Wales Institute Cardiff where she currently combines teaching duties in parallel with extended research activities. She has a breadth of teaching expertise and experience across a variety of subject areas and audiences within both the private and public sector. Her research within the information systems domain has been published widely in both European and international arenas.

Paul Beynon-Davies is currently Professor of Business Informatics at Cardiff Business School, Cardiff University. Before taking up an academic post, Professor Beynon-Davies worked for several years in the information systems industry in the UK. He still regularly acts as a consultant to the public and private sector particularly in the area of info-communications technology (ICT) and its impact on organizational performance. He has researched and published widely in the field having 11 books and over 60 academic papers to his name. Professor Beynon-Davies has engaged in a number of projects related to the impact of ICT on the economic, social and political spheres, both in the UK and Europe.